WMO First Mile Data Collection Guide

World Meteorological Organization

Date: 2026-06-11

Version: 0.1

Document status: DRAFT

Document location: https://wmo-im.github.io/first-mile-guide/first-mile-guide/first-mile-guide-DRAFT.html

Standing Committee on Information Management and Technology (SC-IMT)[1]

Commission for Observation, Infrastructure and Information Systems (INFCOM)[2]

Table of Contents

Chapter 1. Scope

[1] https://community.wmo.int/governance/commission-membership/commission-observation-infrastructures-and-information-systems-infcom/commission-infrastructure-officers/infcom-management-group/standing-committee-information-management-and-technology-sc-imt

[2] https://community.wmo.int/governance/commission-membership/infcom

Chapter 2. Conformance

2.1. Conformance Classes

2.1.1. Requirements Class: Core Data Model

2.1.2. Requirements Class: Standard Name Binding

2.1.3. Requirements Class: MQTT Protocol Binding

2.1.4. Requirements Class: Data Sender

2.1.5. Requirements Class: Data Receiver

Chapter 3. Normative references

Chapter 4. Terms, definitions and abbreviations

4.1. Terms and definitions

4.1.1. First mile

4.1.2. Host device

4.1.3. Observer device

4.1.4. Observation

4.1.5. Parameter definition

4.1.6. Cell method

4.1.7. Cell period

4.1.8. Empty value

4.1.9. Standard name

4.1.10. Namespace

4.2. Abbreviations

- CF
- HMEI
- INFCOM
- MQTT
- protobuf
- QoS
- SC-IMT
- WMO

Chapter 5. Conventions

5.1. Schema notation (Protocol Buffers v3)

5.2. Requirements, recommendations and permissions

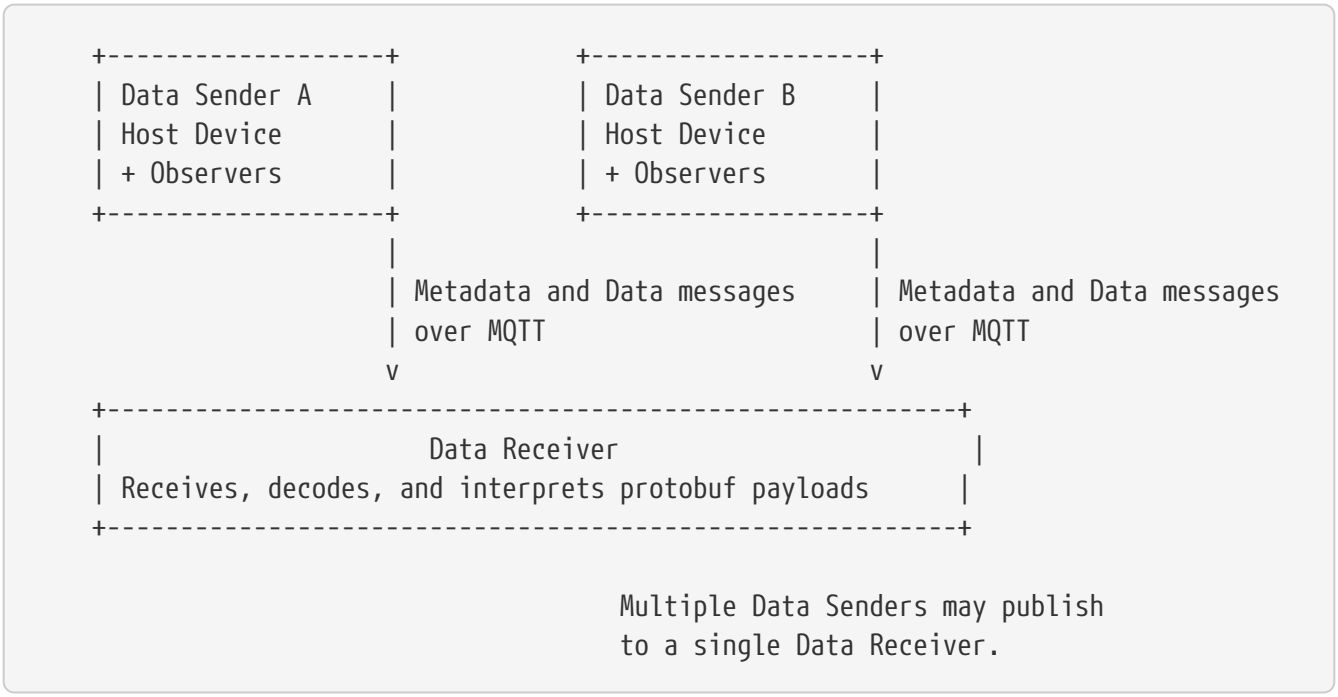
5.3. Namespace and identifier conventions

Chapter 6. Overview (informative)

This clause provides a high-level view of the First-Mile Data Collection Guide. Communication is strictly unidirectional, with the Data Sender transmitting protobuf payloads over MQTT to the Data Receiver. The two top-level payloads are the `Data` message, which carries Observation content, and the `Metadata` message, which carries the descriptive context needed to interpret those observations.

6.1. System architecture

The Data Sender represents the remote station or logger through a Host Device and may describe attached sensors as Observer Device entries. The `Metadata` message carries the Host Device, Observer Device, Parameter Definition, and namespace information, while the `Data` message carries one or more Observation instances that refer to those Parameter Definition identifiers. Detailed message structure is described in Clause 7, standard name binding in Clause 8, and MQTT protocol binding in Clause 9.



6.2. Message flow

In typical operation, the `Metadata` message is sent when the Data Sender is initialized and whenever the station configuration changes, and the `Data` message is sent as observations become available. This separates relatively stable descriptive information from frequently changing measurement values and supports both individual and batched Observation delivery. Where a measurement is unavailable, the value is represented explicitly using `emptyValue` rather than being inferred from zero or omission.

6.3. Conformance class summary

Conformance class	Summary
Core Data Model	Covers the protobuf structure and semantics of the `Data` message, `Metadata` message, Observation content, devices, Parameter Definition content, and explicit missing values.
Standard Name Binding	Covers how Parameter Definition entries associate parameters with controlled standard-name vocabularies and namespaces.
MQTT Protocol Binding	Covers how protobuf payloads are conveyed over MQTT in the unidirectional exchange between Data Sender and Data Receiver.
Data Sender	Covers the sender-side behavior for constructing and publishing `Metadata` message and `Data` message payloads.
Data Receiver	Covers the receiver-side behavior for accepting, decoding, and interpreting payloads received from the Data Sender.

Chapter 7. Data model

7.1. Overview

The data model defines the structure and organization of information transmitted from the Data Sender to the Data Receiver using Protocol Buffers (Protobuf) as the Data Format. The data model is formally specified in the protobuf schema `firstmile.proto`.

The data model supports two primary use cases:

1. **Transmission of observation data** - The Data Sender transmits measurement values collected from sensors
2. **Transmission of metadata** - The Data Sender transmits information about the monitoring site, devices, and parameter definitions

These use cases are supported by two top-level message types: the `Data` message containing observations, and the `Metadata` message containing configuration and contextual information. Both messages are defined in a single protobuf schema and transmitted as independent payloads over MQTT. The `Metadata` message provides the necessary context for interpreting the `Data` message, including descriptions of devices and parameters.

7.1.1. Communication pattern

The communication follows a unidirectional pattern where data flows from the Data Sender to the Data Receiver. The Data Sender may transmit messages in the following patterns:

- **Data-only transmission** - Frequent transmission of `Data` messages containing current or batched observations
- **Metadata transmission** - Transmission of `Metadata` messages when the configuration changes or upon initialization

The Data Sender should transmit the `Metadata` message at least once after initialization and whenever the configuration of devices or parameter definitions changes. The `Data` message may be transmitted as frequently as needed to deliver observations. The Data Receiver can only intepret data correctly if it has received the relevant `Metadata` message that defines the devices and parameters being observed. Therefore, it is recommended to transmit the `Metadata` message at least once before transmitting any `Data` messages.

7.1.2. Message hierarchy

The data model organizes information hierarchically:

- **Messages** - Top-level containers (`Data` and `Metadata`) that are transmitted as independent payloads
- **Devices** - Descriptions of physical equipment including the Host Device (data sender) and Observer Devices (sensors)
- **Observations** - Measurement data points consisting of a parameter identifier, timestamp, and

one or more values

- **Parameter definitions** - Metadata describing what is being measured, including units, aggregation methods, and standard name mappings. Parameter definitions also describe how observations are structured in `Data` messages.
- **Values** - The actual measurement data, supporting multiple numeric types, strings, booleans, and explicit representation of missing data

Each `Observation` references a `ParameterDefinition` by identifier. Each `ParameterDefinition` contains one or more `Parameter` specifications that describe the measured variable, its unit, temporal aggregation method, and which device performs the measurement.

7.1.3. Design principles

The data model is designed according to the following principles:

- **Efficiency** - Optimized for bandwidth-constrained environments but focusing on IP communication rather than extremely low-level binary formats
- **Extensibility** - Support for multiple value types and flexible parameter definitions
- **Self-description** - Metadata provides complete context for interpreting observations
- **Explicit missing values** - Clear distinction between unavailable data and zero/false measurements
- **Batch transmission** - Multiple observations may be included in a single `Data` message
- **Standard name interoperability** - Mapping to external naming conventions through namespace definitions

7.2. Messages

7.2.1. Metadata message

The `Metadata` message provides configuration and contextual information about the Data Sender, including descriptions of devices and the parameters they measure. This message shall be transmitted at least once after Data Sender initialization and whenever the configuration changes.

The `Metadata` message contains the following fields:

Field	Type	Cardinality	Description
host	HostDevice	required	Description of the Data Sender device (datalogger or station)
observers	ObserverDevice	repeated	Descriptions of individual sensor devices connected to the host
parameterDefinitions	ParameterDefinition	repeated	Definitions of the parameters used in observations

Field	Type	Cardinality	Description
namespaces	map<string, string>	optional	Mapping of namespace prefixes to their corresponding URIs for standard name resolution

The `namespaces` field provides the mapping between namespace keys used in `Parameter.standardNames` and their full URIs or identifiers. For example, mapping the key `"cf"` to the URI `"https://vocab.nerc.ac.uk/collection/P07/current/"` enables the use of Climate and Forecast (CF) convention standard names.

7.2.2. Data message

The `Data` message is the primary container for transmitting observation data from the Data Sender to the Data Receiver. This message may be transmitted as frequently as needed to deliver current or batched observations.

The `Data` message contains the following fields:

Field	Type	Cardinality	Description
observations	Observation	repeated	Zero or more observation measurements

A `Data` message may contain multiple `Observation` messages, enabling efficient batch transmission of measurement data. Each observation is independent and contains its own timestamp and parameter reference.

NOTE The Data Sender may transmit `Data` messages containing observations that reference parameter definitions not yet received by the Data Receiver. In this case, the Data Receiver shall buffer or queue such observations until the corresponding `Metadata` message is received.

7.3. Devices

7.3.1. Host device

The `HostDevice` message describes the Data Sender device itself, typically a datalogger or observation station.

The `HostDevice` message contains the following fields:

Field	Type	Cardinality	Description
name	string	required	Name or model designation of the host device
location	Location	optional	Geographic location of the host device

Field	Type	Cardinality	Description
url	string	optional	URI reference to additional metadata about the device
serialNumber	string	optional	Manufacturer's serial number of the device
firmwareVersion	string	optional	Firmware version of the device, typically in format such as "5.1" or "2.7.1-alpha"

The `name` field should contain a human-readable identification of the device, such as its model name or type designation.

7.3.2. Observer device

The `ObserverDevice` message describes an individual sensor or measurement device connected to the Host Device.

The `ObserverDevice` message contains the following fields:

Field	Type	Cardinality	Description
id	uint32	required	Unique identifier for this observer within the context of this host
name	string	required	Name or model designation of the observer device
location	Location	optional	Geographic location of the observer device
url	string	optional	URI reference to additional metadata about the device
serialNumber	string	optional	Manufacturer's serial number of the device
firmwareVersion	string	optional	Firmware version of the device

The `id` field shall be unique among all observers associated with a single host device. This identifier is used by the `DeviceRef` message to associate parameters with specific observer devices.

IMPORTANT	Observer device identifiers shall remain stable across Data Sender restarts and metadata updates to ensure consistent parameter-to-device associations.

7.3.3. Location and reference surfaces

The `Location` message specifies the geographic position of a device (either host or observer).

The `Location` message contains the following fields:

Field	Type	Cardinality	Description
latitude	double	optional	Latitude in decimal degrees, range -90 to +90
longitude	double	optional	Longitude in decimal degrees, range -180 to +180
heightMeter	double	required	Height in meters above the reference surface
referenceSurface	ReferenceSurface	required	The reference surface for the height measurement

The `latitude` and `longitude` fields may be omitted when only the local height of a device is known or relevant. The `heightMeter` field shall always be provided together with a `referenceSurface` specification.

The `ReferenceSurface` enumeration defines the following values:

Value	Description
REFERENCE_SURFACE_UNSPECIFIED	Reserved default value. Shall not be explicitly set by Data Senders.
MSL	Mean Sea Level - the average height of the ocean's surface
GEOID	Geoid - an equipotential surface of the Earth's gravity field
GL	Ground Level - the local ground or surface level at the site
REFERENCE_ELLIPSOID	Reference Ellipsoid - a mathematically defined surface approximating the Earth's shape (e.g., WGS84)
PRESSURE_1000_HPA	The 1000 hPa pressure level in the atmosphere

NOTE The choice of reference surface is determined by the user based on the measurement context and requirements. For example, meteorological observations of air temperature typically use height above ground level (GL), while atmospheric pressure observations may reference mean sea level (MSL).

7.4. Observations and values

7.4.1. Observation

The `Observation` message represents a single measurement data point collected at a specific time.

The `Observation` message contains the following fields:

Field	Type	Cardinality	Description
parameterDefinitionId	uint32	required	Identifier referencing a `ParameterDefinition` in the `Metadata` message

Field	Type	Cardinality	Description
time	google.protobuf.Timestamp	required	The timestamp when the observation was made, in UTC
values	Value	repeated	One or more measurement values

The `parameterDefinitionId` field shall reference a `ParameterDefinition.id` from the `Metadata` message. The number and type of values in the `values` field shall correspond to the number of parameters defined in the referenced `ParameterDefinition`.

The `time` field uses the standard `google.protobuf.Timestamp` format, representing time as seconds and nanoseconds since the Unix epoch (1970-01-01T00:00:00Z).

NOTE

An observation may contain multiple values when the referenced `ParameterDefinition` contains multiple parameters. This approach enables grouping values that are measured at the same timestamp, requiring the timestamp to be transmitted only once for multiple values. This reduces payload size, which is particularly beneficial in bandwidth-constrained environments.

For example, a parameter definition for "Internal Parameters" might include both supply voltage and internal temperature, requiring two values per observation but sharing a single timestamp.

7.4.2. Value types

The `Value` message is a polymorphic container that can represent measurement data of different types. The message uses a `oneof` field, meaning exactly one of the following value types shall be set:

Field	Type	Description
floatValue	float	32-bit floating point number
doubleValue	double	64-bit floating point number
intValue	int32	32-bit signed integer
unsignedIntValue	uint32	32-bit unsigned integer
int64Value	int64	64-bit signed integer
unsignedInt64Value	uint64	64-bit unsigned integer
stringValue	string	Text string value
boolValue	bool	Boolean true/false value
emptyValue	google.protobuf.Empty	Indicates missing or unavailable data

The choice of value type should be appropriate for the parameter being measured. Numeric measurements typically use `doubleValue` or `floatValue`, while categorical observations may use `stringValue`. Boolean parameters (such as alarm states) use `boolValue`.

NOTE

The `stringValue` field is intended for text data only and shall NOT be used to transmit:

- Binary data, including base64-encoded binary blobs
- Serialized objects or data structures
- Structured data formats such as JSON, XML, or similar encodings

Binary data transmission and complex structured data are outside the scope of this standard.

7.4.3. Missing and error values

When a measurement value is unavailable due to sensor malfunction, communication error, or other causes, the value shall be represented using the `emptyValue` field of type `google.protobuf.Empty`.

IMPORTANT

Missing or unavailable measurement values shall NOT be represented as:

- Numeric zero (0)
- NaN (Not a Number)
- Omission from the `values` array
- Any other numeric sentinel value

The `emptyValue` field shall be used to explicitly and unambiguously indicate missing data.

This approach ensures clear distinction between actual measurements of zero and missing measurements, which is critical for correct data interpretation.

When an `Observation` references a `ParameterDefinition` containing multiple parameters, the values in the `values` array shall be provided in the same order as the parameters are defined in the `ParameterDefinition.parameters` field. It is not permitted to omit a value from the middle of the array, as this would cause misalignment with the parameter definitions. If a value is unavailable, the `emptyValue` field shall be used at that position to maintain correct alignment.

7.5. Parameter definitions

A `ParameterDefinition` groups one or more related measurement parameters that share common context. Each parameter definition has a unique identifier that is referenced by observations. Parameters are typically grouped when they are measured at the same timestamp, allowing multiple values to share a single timestamp in observations and thus reducing payload size.

The `ParameterDefinition` message contains the following fields:

Field	Type	Cardinality	Description
id	uint32	required	Unique identifier for this parameter definition
description	string	required	Human-readable description of this parameter definition
parameters	Parameter	repeated	One or more parameter specifications

The `id` field shall be unique within the scope of a single `Metadata` message. This identifier is used by the `Observation.parameterDefinitionId` field to associate observations with their parameter specifications.

The `parameters` field contains one or more `Parameter` messages describing individual measurable variables. When multiple parameters are grouped in a single definition, each observation referencing this definition shall provide values in the same order as the parameters are defined.

7.5.1. Parameter

The `Parameter` message specifies a single measurable variable, including its name, unit of measurement, aggregation method, and associated device.

The `Parameter` message contains the following fields:

Field	Type	Cardinality	Description
longName	string	required	Descriptive name of the parameter (e.g., "Air Temperature", "Wind Speed")
unit	string	required	Unit of measurement (e.g., "°C", "m/s", "hPa")
standardNames	map<string, string>	optional	Mapping of namespace prefixes to standard names in external naming conventions
device	DeviceRef	required	Reference to the device (host or observer) that measures this parameter
cellMethod	CellMethod	required	Statistical or aggregation method applied to the measurement
cellPeriodSeconds	uint32	required	Period of aggregation in seconds (0 for instantaneous point measurements)

The `standardNames` field enables mapping to external naming conventions such as Climate and Forecast (CF) standard names. This allows parameters to be referenced by different standard parameter types according to the target application of the device using the protocol. The Data Sender is not bound to a specific standard but may include mappings to multiple standards as appropriate for the intended use case. The keys in this map shall correspond to namespace prefixes defined in `Metadata.namespaces`.

The `device` field specifies which device (either the host or a specific observer) performs this measurement.

7.5.2. Cell method

The `cellMethod` field specifies how measurement data is aggregated or processed over the cell period. The `CellMethod` enumeration defines the following values:

Value	Description
CELL_METHOD_UNSPECIFIED	Reserved default value. Shall not be explicitly set by Data Senders.
POINT	Instantaneous value at a specific point in time (no aggregation)
SUM	Sum of values over the cell period
MAXIMUM	Maximum value observed during the cell period
MINIMUM	Minimum value observed during the cell period
MEAN	Mean (average) value over the cell period
MEDIAN	Median value over the cell period
MODE	Most frequently occurring value
VARIANCE	Statistical variance within the cell period
STANDARD_DEVIATION	Standard deviation within the cell period
MAXIMUM_ABSOLUTE_VALUE	Maximum of absolute values
MINIMUM_ABSOLUTE_VALUE	Minimum of absolute values
MEAN_ABSOLUTE_VALUE	Mean of absolute values
MEAN_OF_UPPER_DECILE	Mean of the upper tenth of the value distribution
MID_RANGE	Average of maximum and minimum values
RANGE	Absolute difference between maximum and minimum values
ROOT_MEAN_SQUARE	Root mean square (RMS) value
SUM_OF_SQUARES	Sum of squared values

NOTE When `cellMethod` is set to `POINT`, the measurement represents an instantaneous value and `cellPeriodSeconds` shall be 0. For all other cell methods, `cellPeriodSeconds` shall specify the duration over which the aggregation is performed.

7.5.3. Cell period

The `cellPeriodSeconds` field specifies the time period in seconds over which the cell method is applied.

- When `cellMethod` is `POINT`, the value shall be 0, indicating an instantaneous measurement.

- For all other cell methods, the value shall be greater than 0, indicating the duration of the aggregation period.

For example: * A 30-second mean temperature would have `cellMethod` = `MEAN` and `cellPeriodSeconds` = 30 * An instantaneous wind direction would have `cellMethod` = `POINT` and `cellPeriodSeconds` = 0 * A 5-minute maximum wind gust would have `cellMethod` = `MAXIMUM` and `cellPeriodSeconds` = 300

7.5.4. Device reference

The `DeviceRef` message associates a parameter with the device that measures it. This message uses a `oneof` field to reference either the host device or a specific observer device.

The `DeviceRef` message contains exactly one of the following fields:

Field	Type	Description
host	google.protobuf.Empty	References the host device; the presence of this field (even though empty) indicates the parameter is measured by the host
observerId	uint32	References an observer device by its `ObserverDevice.id` value

The `oneof` constraint ensures that exactly one of these fields is set, unambiguously identifying which device performs the measurement.

When the `host` field is present, the parameter is measured by the Host Device itself (such as internal voltage or temperature of a datalogger). When the `observerId` field is set, it shall match the `id` of an `ObserverDevice` defined in the `Metadata` message.

Chapter 8. Standard name binding

8.1. Namespace declaration

8.2. Standard name assignment

8.3. Recommended: CF Conventions binding

Chapter 9. Protocol binding

9.1. Overview

9.2. MQTT version requirements

9.3. Topic structure

9.3.1. Topic pattern

9.3.2. Level definitions and allowed values

9.4. Quality of service

9.4.1. Metadata messages

9.4.2. Data messages

Chapter 10. Data sender

10.1. Metadata publication

A key feature of the First Mile Format, is the structured inclusion of metadata. This enables

Benefit	Definition	Required in implementation
Machine Translation	Received data can be correctly assigned to observations in database without human interpretation/intervention	Standard Names
Stranger Interpretation	Siting (particularly location and height of observations) and equipment can be deduced from data, enabling assessment of observation quality without access to other information	Location, URL (for additional information)
Efficient Communication	Metadata is not included in every message (reducing bandwidth requirements), only to ensure efficient processing by the Receiver	Transmission Triggers

10.1.1. Mandatory fields

To enable evolution of the First Mile Standard, while maximising backward compatability, the Protobuf **REQUIRED** keyword is not used. However, omission of key values can render the other included information unusable as it is ambiguous as to how it is related to the other content.

Message	Field	Justification
HostDevice	name	Enables the identification of the end node
ObserverDevice	id	Enables parameters (observation data) to be matched to the sensing/observing/measuring device
ParameterDefinition	id	Enables content from Data messages to be interpreted
Parameter	unit	Critical to hub interpretation of received data
Parameter	DeviceRef	Enables sensors/observers to be matched to their data

If machine to machine translation is expected, then the following following additional fields must be included

Message	Field	Justification
Parameter	standardNames	Enables machine to machine translation of definition, irrespective of parameter name
Parameter	CellMethod	If any data processing is done at the edge, this enables for example the mean and standard deviation to be differentiated (as they will have the same standardNames, units)

10.1.2. Transmission triggers

The hub is unable to process any Data Messages without a current Metadata Message.

As a minimum, a new/updated Metadata Message must be transmitted whenever the Metadata changes. This is particularly important when:

- Sensors are added or removed
- New Observations (or statistics on existing observations, for example max/min values) are provided.

To balance network transmission costs for Metadata Messages with impost of storing Data messages until the relevant Metadata message is received and the Data Messages can be understood, recommended practice is (re)sending of the Metadata Message:

- on power-on/restart of hub
- on a "heartbeat" schedule of at least once per day.

10.1.3. Retained messages

The Data Messages cannot be understood without their associated Metadata Message. To meet Archiving requirements [1] in most jurisdictions the Data Messages, and associated Metadata Messages must both be retained. Typically, this can be a one (Metadata Message) to many (Data Messages) relationship

[1] For Example WMO 1238 (2023) 2.5.5

Chapter 11. Data publication

The First Mile Format enables efficient communication of Data:

- the message is a compact/efficient binary format.
- only the Observation/Data values are transmitted [as the less often transmitted Metadata Message carries the information to understand the values]

11.1. Mandatory fields

Every Data message must contain at least:

parameterDefinitionId	Enables the Data to be matched to the definition from the Metadata Message
time	
(at least one) value	

11.2. Observation timestamps

The Data (Observation) Timestamp uses the google.protobuf.Timestamp structure for each observation.

```
message Timestamp {
    int64 seconds = 1; // Seconds since UTC (Unix) epoch (1/1/1970)
    int32 nanos = 2; // non-negative nanosecond fractions
}
```

The structure enables date/time resolutions to nanoseconds for observations.

11.3. Device identification

The Data Message does not contain any Station/Device information in the message. Instead, the MQTT topic hierarchy NodeID (Level 4) would be used for Station/Device Identification and should be archived with the Data Message.

Chapter 12. Data receiver (structure TBD)